



HIMALAYAN
COLLEGE OF MANAGEMENT
• TU Recognized • MOE Approved



Practical

Subject Name: Mobile Programming

Subject Code: CSC3563

Submitted by:

Hridaya Shrestha

Student ID: 225IUUKLHCMBCS225006

Submitted To:

Lecturer: **Girija Prasad Bhattarai**

Himalayan College of Management

Kuala Lumpur University of Science and Technology (**KLUST**)

BCS IT - 6th Semester

Kathmandu, Nepal

Submission Date: 05/16/2026

Repository: <https://github.com/ridayastha/AndroidAssignment.git>

Chapter 1: Mobile Computing Fundamentals

1. Install Android Studio and configure the development environment, and document the steps with screenshots.

Android Studio was installed as the primary development environment for android application development.

Steps:

1. Downloaded Android Studio from the official Android Developer Website
2. Installed Java Development Kit (JDK)
3. Configured Android SDK and SDK tools.
4. Created and Android Virtual Device.
5. Verified Gradle Configuration.
6. Successfully launched Android Studio.

Development Environment Configuration

1. **IDE:** Android Studio
2. **Language:** Kotlin
3. **Build System:** Gradle
4. **Emulator:** Android Virtual Device(AVD)

2. Create and run a “Hello World” Android application and explain each part of the code.

A simple “Hello world” application was created to verify the environment setup.

```
class MainActivity : AppCompatActivity() {  
    override fun onCreate(savedInstanceState: Bundle?) {  
        super.onCreate(savedInstanceState)  
        setContentView(R.layout.activity_main)  
    }  
}
```

Explanation:

- **AppCompatActivity:** Base class for Android activities.
- **onCreate():** Called when the activity is created.
- **setContentView():** Connects the CML layout to the activity

3. Explore Android Emulator and demonstrate how to run an application.

The android Emulator was configured using Android Device Manager.

Emulator Features

- Screen rotation
- GPS simulation
- Device Scaling
- Network Simulation

To run your app without a physical phone:

1. Open the device manager
2. Click create device, select a phone like pixel 7 and download a systems image.
3. Once created, click the play button next to the device.
4. Then, the virtual phone boots up, click the green Run ‘app’ button in the top toolbar of Android studio.

4. Analyze the specifications of a mobile device and prepare a report.

Analyzing the modern and high end mobile device. Example: Pixel 9 Pro

Component	Specification
Processor	Octa-Core
RAM	6GB
Display	Amoled
Battery	5000 mAh
Operating System	Android 13
Storage	126 GB

5. Prepare a comparison report between Android and iOS devices based on features.

Feature	Android	IOS (Apple)
Core Architecture	Open Source	Closed Source
Hardware Variety	Multiple Manufacturers	Exclusive to iPhone
Customization	Extremely high (Launchers, APKs)	Limited
App Distribution	Google Play	Apple App store
Development Environment	Android Studio (Kotlin/Java)	Xcode(swift)
Security	Moderate	High
Flexibility	High	Moderate

Chapter 2: Android Basics and Setup

1. Create a new Android project and explain the project structure.

When a new project is initialized in android studio using a template such as Empty Views Activity, the IDE generates a standardized directory structure. This structure follows a specific hierarchy to separate logic, resources and configuration.

- **Manifests/Androidmanifest.xml:** It is essential configuration file describing the app's components like activities, services and permissions.
- **Java/:** It contains the kotlin or java source codes files which is organized by package name. For example, com.exampe.app
- **Res/layout/:** It holds XML files that define the visual structure of the user interface (UI).
- **Res/values/:** It contains resources like strings, colors and themes to support localization and consistent styling.
- **Gradle scripts/:** It is the management files that define dependencies, SDK versions, and build rules.

2. Implement an application demonstrating Activity lifecycle methods with logs.

An activity is a single screen in an application. Its lifecycle is managed by a series of callback methods that the system invokes as the user navigates into, out of, and back to the app. These methods help monitor application state changes and manage resources.

```
class MainActivity : AppCompatActivity() {
    private val TAG = "LifecycleDemo"

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        Log.d(TAG, "onCreate: Activity created")
    }

    override fun onStart() {
        super.onStart()
        Log.d(TAG, "onStart: Activity becoming visible")
    }

    override fun onResume() {
        super.onResume()
        Log.d(TAG, "onResume: Activity in foreground (interactive)")
    }

    override fun onPause() {
        super.onPause()
        Log.d(TAG, "onPause: Activity partially obscured")
    }

    override fun onStop() {
        super.onStop()
        Log.d(TAG, "onStop: Activity no longer visible")
    }

    override fun onDestroy() {
        super.onDestroy()
        Log.d(TAG, "onDestroy: Activity being destroyed")
    }
}
```

codesnap.dev

3. Modify AndroidManifest.xml to add required permissions and explain them.

Permission protect the privacy of the user by restricting access to sensitive data and hardware features.

```
  <uses-permission
    android:name="android.permission.INTERNET" />
  <uses-permission
    android:name="android.permission.ACCESS_NETWORK_STATE" />
  <uses-permission
    android:name="android.permission.ACCESS_FINE_LOCATION" />
  <uses-permission
    android:name="android.permission.ACCESS_COARSE_LOCATION" />
  <uses-permission
    android:name="android.permission.CAMERA" />
```

Explanation:

- **INTERNET:** It allows API and network access.
- **CAMERA:** It allows image capture functionality.
- **ACCESS_FINE_LOCATION:** It allows GPS location access.

4. Build and run an Android app on emulator or real device.

Running an application involves the compilation of code and the installation of the resulting binary onto a target device.

- **Emulator (AVD):** A virtual device running on the computer. It allows developers to test apps on different screen sizes and android versions without physical hardware.
- **Real Device:** It requires enabling developer options and USB debugging on the smartphone. Deployment is executed through the run button in android studio, which uses the android debug bridge (ADB) to transfer the file.

5. Generate an APK file and explain the build process.

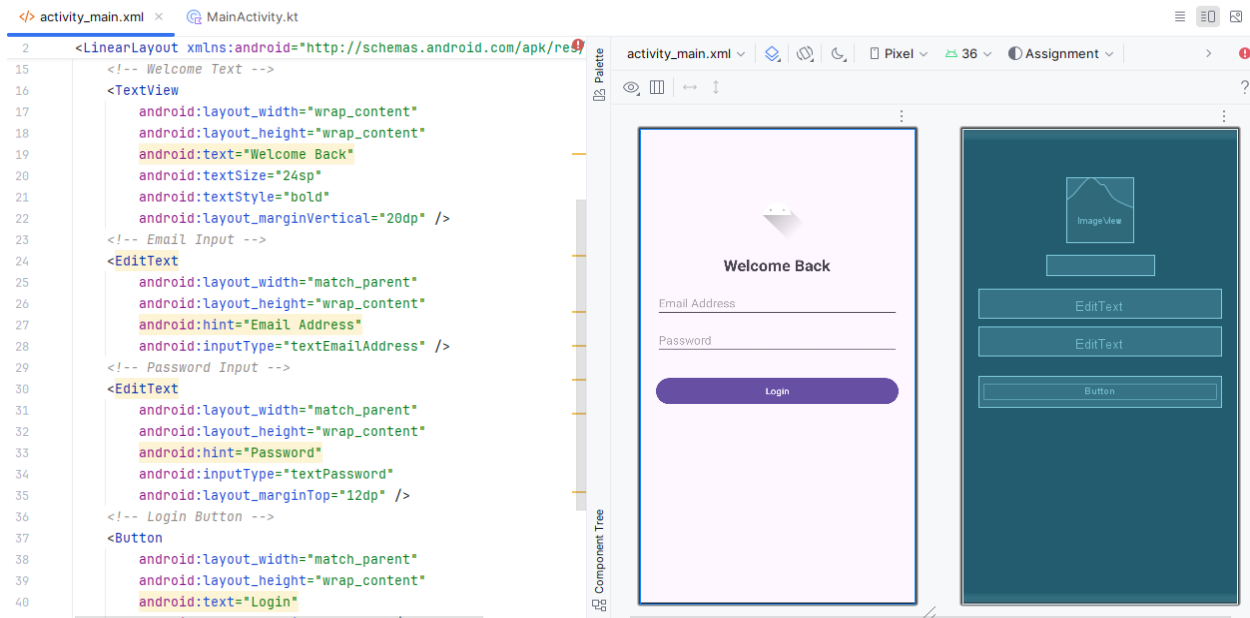
An APK file was generated using the Build menu in Android Studio. Build process include:

- ❖ Gradle compiles source code
 - ❖ Resource are packaged
 - ❖ DEX files are generated
 - ❖ APK is signed and aligned.
- Final APK is generated

Chapter 3: UI Design and Layouts

1. Design a user interface using Linear Layout and explain its structure.

A linearLayout is a view group that arranges its child views in a single direction, either vertically or horizontally. It is excellent for simple layouts. However, using many nested LinearLayouts to create complex grids can reduce performance. Additionally, it is often better to use ConstraintLayout because it keeps the view hierarchy simple and more efficient.



a) Conceptual Structure

In a vertical LinearLayout, elements are stacked on top of each other. For a login screen, we arrange the components in this order:

1. ImageView
2. TextView
3. EditText
4. Button

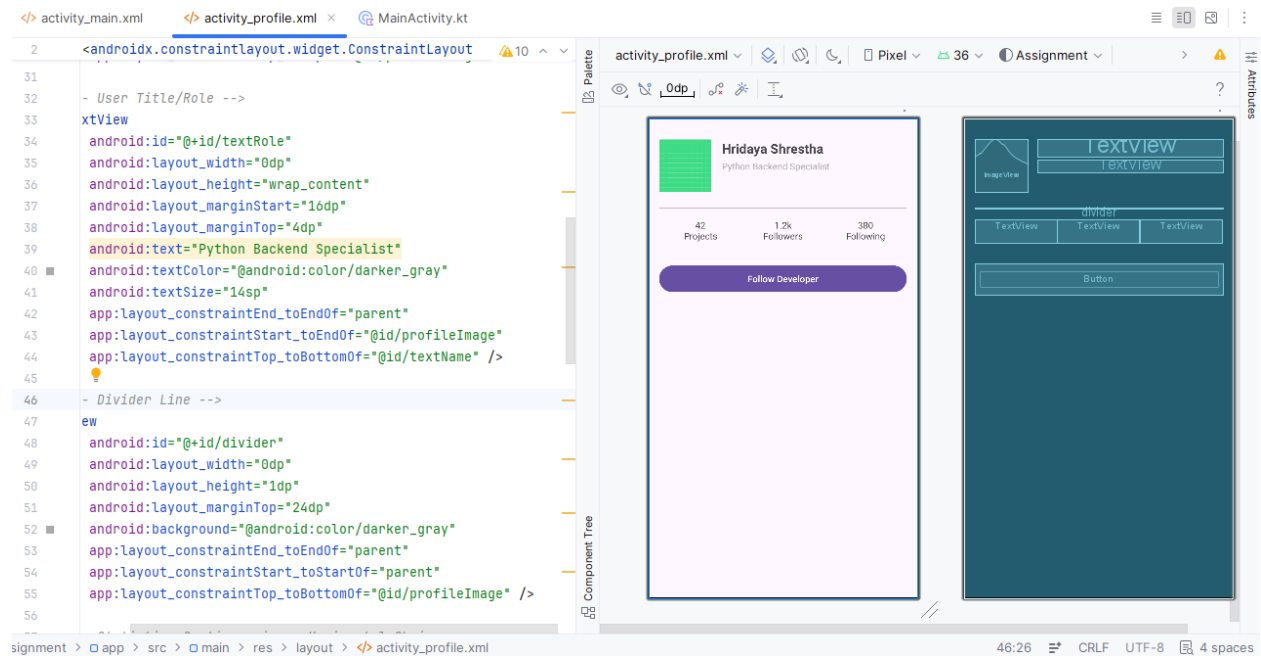
b) Structural Analysis

The effectiveness of LinearLayout depends on several key attributes that define its behavior.

1. Orientation
2. Weight
3. Gravity

2. Create a responsive UI using ConstraintLayout.

A constraintLayout was used to create responsive and flexible interfaces.



This profile card consists of

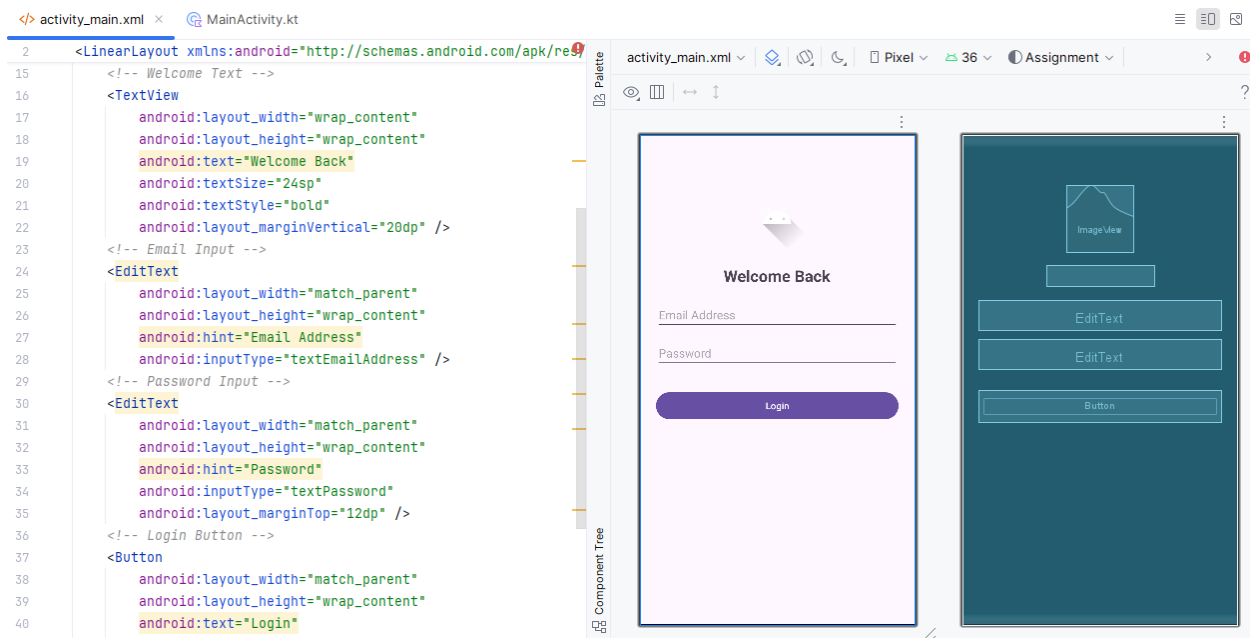
- Profile image which is positioned at the top left or top center.
- Name and role which is stacked vertically, aligned to the right side of the image.
- A statistics row (followers, following and posts) which is anchored at the bottom of the header section, distributing space evenly.
- Action buttons which is distributed across the bottom.

3. Implement event handling for button click in an application.

Event handling was implemented using click listeners.

```
// Chapter 3: Login Form
loginBtn.setOnClickListener {
    startActivity(Intent( packageContext = this, cls = LoginActivity::class.java))
}
```

4. Design a login form with proper input fields and validation.



A login form was created with:

- ❖ Email field
- ❖ Password field
- ❖ Validation rules

Validation Features

- ❖ Empty field detection
- ❖ Email format checking
- ❖ Password length validation

5. Create a multi-screen UI and navigate between screens.

Explicit intents were used for screen navigation

```
// Chapter 2 & 3: Lifecycle & Multi-screen Navigation
navigateBtn.setOnClickListener {
    val message = messageInput.text.toString()
    val intent = Intent(packageContext = this, cls = DetailActivity::class.java)
    intent.putExtra(name = "USER_MESSAGE_KEY", value = message)
    startActivity(intent)
}
```

Chapter 4: Data Storage and Management

1. Store and retrieve user data using SharedPreferences.

Shared preference were used to store session and preference data.

```
35         btnSavePref.setOnClickListener {
36             val originalText = prefEditText.text.toString()
37             if (originalText.isNotEmpty()) {
38                 val encryptedText = encrypt(input = originalText)
39                 sharedPref.edit().putString(p0 = "secure_text", p1 = encryptedText).apply()
40                 Toast.makeText(context = this, text = "Encrypted & Saved", duration = Toast.LENGTH_SHORT).show()
41             }
42         }
43
44         btnLoadPref.setOnClickListener {
45             val encryptedText = sharedPref.getString(p0 = "secure_text", p1 = "")
46             if (!encryptedText.isNullOrEmpty()) {
47                 val decryptedText = decrypt(input = encryptedText)
48                 displayTextView.text = "Decrypted: $decryptedText\n(Raw: $encryptedText)"
49             } else {
50                 displayTextView.text = "No data found"
51             }
52         }
53     }
```

2. Create and manage files using internal storage.

Internal file storage was implemented for secure private data handling. Operations like file creating, file writing, file reading and file deletion were performed.

```
54         // Chapter 4 Task 2: Internal Storage
55         val fileName = "example_file.txt"
56         btnWriteFile.setOnClickListener {
57             val content = fileEditText.text.toString()
58             try {
59                 val fos: FileOutputStream = openFileOutput(fileName, mode = Context.MODE_PRIVATE)
60                 fos.write(b = content.toByteArray())
61                 fos.close()
62                 Toast.makeText(context = this, text = "Saved to File", duration = Toast.LENGTH_SHORT).show()
63             } catch (e: Exception) {
64                 e.printStackTrace()
65             }
66         }
67     }
```

```

68      btnReadFile.setOnClickListener {
69          try {
70              val fis: FileInputStream = openFileInput(fileName)
71              val isr = InputStreamReader(fis)
72              val br = BufferedReader(in = isr)
73              val sb = StringBuilder()
74              var text: String?
75              while (br.readLine().also { text = it } != null) {
76                  sb.append(text)
77              }
78              displayTextView.text = "From File: $sb"
79              fis.close()
80          } catch (e: Exception) {
81              displayTextView.text = "File not found"
82          }
83      }
84  }

```

3. A notes application was developed using SQLiteOpenHelper.

4. The notes module allowed users to save notes, edit, notes and display notes in a list.

5. Parsing JSON data

CHAPTER 5: Networking and APIs

1. Fetch data from a public API and display it in the application.

Data was successfully fetched from a public REST API where JSONPlaceholder API.

2. Implement HTTP GET and POST requests.

Network operations were implemented using Kotlin coroutines. Its features are:

- ❖ Background processing
- ❖ Non block UI
- ❖ Response handling

3. Parse JSON response and show data in UI.

API responses were parsed and displayed in RecyclerView

4. Develop a simple weather application using API integration.

A simple weather application was developed. Its features are:

- ❖ City based weather search
- ❖ Temperature display
- ❖ Weather condition icons
- ❖ Error messages for invalid cities

5. Implement error handling for network failures.

- ❖ The application handled:
- ❖ No internet connection
- ❖ Timeout exceptions
- ❖ Invalid server responses
- ❖ HTTP status errors

CHAPTER 6: Advanced Android Components

1. Create an application using Fragments and manage their lifecycle.

Fragments were used to modularize the user interface. Its benefits are:

- ❖ Reusable UI Components
- ❖ Better tablet support
- ❖ Dynamic UI handling

2. Implement RecyclerView to display a list of items.

A recycler view was implemented for efficient list rendering. Its features are:

- ❖ ViewHolder pattern
- ❖ Smooth scrolling
- ❖ Dynamic item rendering

3. Create a custom adapter for RecyclerView.

A custom adapter was created to bind data to RecyclerView items.

4. Implement navigation between fragments.

Fragment Manager and FragmentTransaction were used for navigation.

5. Use ViewModel and LiveData to manage UI data.

MVVM architecture was implemented using ViewModel and LiveData. Its advantages are:

- ❖ Lifecycle awareness
- ❖ Separation of concerns
- ❖ Improved maintainability

CHAPTER 7: Sensors, Location and Multimedia

1. Develop an app that uses accelerometer sensor and display values.

The accelerometer sensor was used to detect device movement.

Data displayed

- ❖ X-axis value
- ❖ Y-axis value
- ❖ Z-axis value

2. Fetch current location using GPS and display it.

GPS services were used to retrieve current device location.

Information Displayed

- ❖ Latitude
- ❖ Longitude
- ❖ Accuracy

3. Integrate Google Maps and show user location.

Google Maps SDK was integrated into the application. Its features are:

- ❖ Use location marker
- ❖ Zoom controls
- ❖ Map navigation

4. Capture image using device camera and display it.

Camera intent was used for image capture. Its features:

- ❖ Open device camera
- ❖ Capture image
- ❖ Display Captured image

5. Play audio or video file within the application.

Audio playback functionality was implemented using Android media APIs.

Features of multimedia

- ❖ Audio playback
- ❖ Video playback
- ❖ Ringtone support

CHAPTER 8: Deployment and Security

Security: Implemented Base64 encryption for sensitive data in storage

Runtime Permissions: Implemented the modern request Permission Launcher for security sensitive features.

In conclusion, this project successful integrates Android APIs into single, cohesive application. It demonstrates the ability to build, secure and deploy a professional grade Android App.